基于模拟退火粒子群算法在电网规划中的应用

学　　院　　数学与计算机学院

专业班级　　信计 2002 班

学生姓名　　吴奇灵　　王壮

学　　号　2012010048 2012010019

目 录

一 引 言 3
（一） 问题的描述 4
（二） 问题的分析 4
（三） 文章的结构 4
二 电网优化模型 4
（一） 并网型电网优化模型 5
（二） 以经济性为目标的数学模型 6
（三） 以环保性运行为目标的数学模型 6
（四） 以综合效益运行最优为目标的数学模型 6
三 模拟退火粒子群算法介绍 7
（一） 模拟退火算法 7
（二） 粒子群算法 7
（三） 改进的模拟退火粒子群算法 8
四 问题的设计及结果分析 9
（一） 模拟退火粒子群算法求解的思路和流程 9
（二） 问题的求解和结果分析 11
五 实验总结 14
参考文献 15
附录 16

摘　要：随着现代人们保护环境的意识增强，我国也在朝着绿色化发电方向发展，目前面临着化石燃料短缺和环境污染等客观事实的压力，促使灵活、清洁的分布式能源愈加广泛的应用在微电网中，本文的探究主旨便是微电网的优化运行。首先，基于微电网运行时追求目标不同，成本最低和污染最小是不能同时满足的，本文从经济性、环保性考虑，提出包括光伏发电、风力发电、微型燃气轮机、燃料电池、垃圾焚烧发电和储能装置的微电网模型，建立同时考虑微电网燃料成本、维护管理成本、折旧成本、与公共电网交互成本和污染物处理费用的综合效益目标函数。其次，本文在传统粒子群算法(PSO)的原型基础上，改进了惯性权重的迭代方式，提高了收敛速度和收敛合理性，先是结合了模拟退火算法(SA)，最后，本文根据建立的综合效益目标函数，运用改进的模拟退火粒子群算法，得出更适用于微电网的运行方式。并通过模拟退火粒子群算法、模拟退火粒子群算法和传统粒子群算法两两实验对比，验证了改进后的模拟退火粒子群算法更具有科学性和可行性。

关键字：粒子群算法、模拟退火算法、电网优化、综合效益

一 引　言

随着现代人类对环保意识的加强，以及化石燃料的日益枯竭，传统的以单纯消耗不可再生能源的发电方式已不再是我们考虑的首要方式。我国能源产业化转移行动正在进行，风力、光伏等分布式发电以其得天独厚的优势（包括安全的能源、清洁无污染、高效利用、人力资源占用少等）在我国得到飞速发展。政府当然也是高度重视，并发布多种相关政策支持，更是推动了新能源技术的发展。

微电网作为电力行业改革以及新能源技术领域的重要组成部分，由多种分布式电源(Distributed Generation，DG)、用户负荷、储能装置以及各单元自动控制模块组成。一方面，微电网发挥着 DG 所特有的经济性、环保性、灵活性，提供优质的电能质量；另一方面，微电网具有两种运行模式：1)孤岛运行模式，是一种微电网当作独立的电力系统直接供电给用户的工作模式；2)并网运行模式，旨在让微电网和公共电网协调运行，以防故障以及用电需求不平衡，提高供电可靠性能源利用率，降低停电率和能耗。微电网中分布式电源的形式是丰富多样的，有充分利用大自然能量的光伏、风力发电装置，也有利用如氢气这类清洁能源的微型燃气轮机和燃料电池，还有变

废为宝的垃圾焚烧发电，必要时也有传统化石燃料发电装置，再通过协调储能装置和公共电网的交互可以提高用电的可靠性与稳定性。

（一） 问题的描述

微电网可以理解为是一种小型的电力系统网络，并且与公共电网一样具有发输及供配电的功能，并网运行时，又可以以公共电网的视角将微电网看成是一个可控的微电源，并入公共电网之后，可以有效减少系统经济成本、合理调度负荷分配，提高该系统的经济性与稳定性，具体对微电网中各个分布式单元的数学模型进行分析，是合理控制优化微电网内部能量分配的基础。

诚然，多种多样分布式电源的组合及出力方式可以解决不同的用电需求，而在保证用电可靠性稳定性的前提下，科学的配比使其达到经济效益最大化，环境污染最小化则是目前探讨的重点。当前多国和各地公司也对微电网优化运行进行了大量的实验研究，投入较多的时间精力和资金，因此，分析和探讨更合理的微电网结构、更适用于现代工业生产和生活的运行方式将对人们大有裨益。

（二） 问题的分析

建立并网型微电网优化与策略数学模型，在风力发电和光伏发电全力输出功率的情况下，在保障经济效益的同时，追求绿色可持续的发展，建立了以综合效益运行最优为目标的数学模型，采用线性权重赋值的方法，找出经济性运行和环保性运行间的平衡点，使寻求的目标函数设计更合理。

将采用改进后的模拟退火粒子群算法求解该问题。为了克服传统粒子群算法易早熟，易陷入局部最优解等缺陷，本文将模拟退火算法和粒子群算法结合，并通过改进惯性权重的迭代方式来提高算法的收敛速度。使得到的微电网优化运行方式更合理。

（三） 文章的结构

本文第二章主要介绍电网优化模型。第三章主要介绍模拟退火粒子群算法。第四章进行问题的设计及结果分析。

二 电网优化模型

微电网需要的优化运行调度和控制策略，与传统的电力系统有着明显不同，与各微型电源的类型及其渗透程度、负荷特性以及电能质量约束条件有关。因此，很有必要对微电网并网模式下的策略进行分析。并网运行方式是指微电网通过公共连接点（PCC）接入大电网，与大电网交换有功/无功功率。当各分布式电源出力之和大于负荷功率时，微电网从大电网出售部分电能；相反，当各分布式电源出力之和小于负荷功率时，微电网从大电网购买部分电能。在并网运行模式下，微电网可

应用电力市场规律灵活地控制各分布式电源的运行，获得更大的经济效益和环境效益。目前，主流的并网型微电网运行策略主要分为 3 种：联络线功率控制、专家策略和调度运行。本文微电网并网时采取的优化调度策略如下具体所述：

（1） 由于风力发电和光伏发电使用自然资源，对燃料没有特殊要求，不会有污染，因此它们优先发电。采用最大功率输出跟踪策略实现发电量的最大化。

（2） 蓄电池在电能盈余或不足的情况下可以起到充放电作用。为保证夏/冬季典型日每天同样的调度过程，满足日前调度。蓄电池在经过一天 24h 周期调度以后，恢复到初始状态。

（3） 在夏季和冬季，由于天气和温度的影响，系统有一定的冷、热负荷需求。为了满足上述负荷的需求，实现梯级用能的目的，在不考虑蓄热和冷热能量外送的情况下，采用微型燃气轮机“以冷定电、以热定电”的工作方式，因此也优先采用微型燃气轮机发电。

（4） 计算完光伏发电、风力发电及微型燃气轮机的各输出功率值后，计算剩余负荷需求。若各分布式电源出力之和大于负荷功率，按分时电价机制将多出的电能出售给主网或存储在蓄电池中。在含光/风/微/燃/蓄或光/风/微/柴/蓄优化调度方式中，在仍不能满足负荷需求的情况下，将 FC 或 DE 发电综合成本与主网售购电综合成本进行比较，优化 FC 或 DE 与主网的出力，使综合成本最小化。在含光/风/微/燃/柴/蓄的调度方式中，可比较前两种光/风/微/燃/蓄或光/风/微/柴/蓄优化调度下 FC 和 DE 的综合成本，按综合成本最优为原则，综合成本更少的分布式电源出力为主，综合成本更多的分布式电源出力为辅。

（一） 并网型电网优化模型

并网型微电网模型一般包括三个基本元素：微电源装置、用户负荷、储能装置。本文模型中储能装置考虑的是较常见的铅酸蓄电池(Lead-acid batteries，BAT)，负荷指生活中的日常负荷需求。并网型微电网结构模型如图 1 所示：

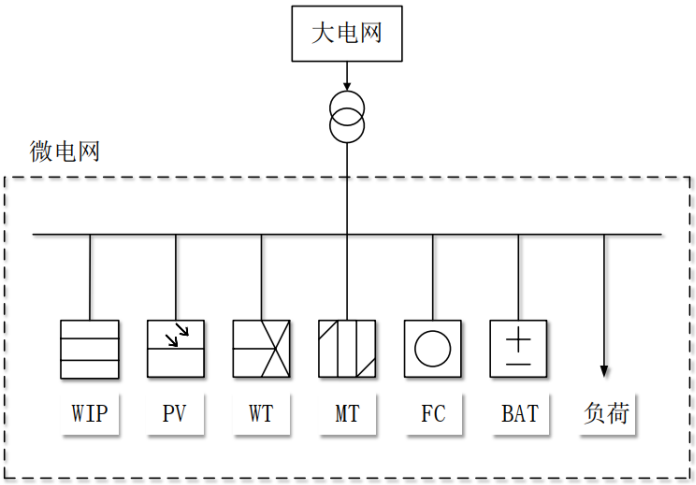


图 1 并网型微电网优化模型

（二） 以经济性为目标的数学模型

参照各地建成的微电网中涉及的经济费用，本文建立的经济性运行成本最低的目标函数，主要考虑整个系统的燃料耗费成本、维护管理成本、设备的折旧损耗以及与公共电网的交互成本。公式如下：

$$\min C_1 = \sum_{t=1}^{T}\left(C_f(t) + C_{ma}(t) + C_{dep}(t) + kC_{grid}(t)\right) \tag{1}$$

式中： C_1 是只考虑微电网经济性运行时的运行成本；$\sum_{t=1}^{T}()$指计算平均到每天的运行成本，T=24，即 1-24 小时的运行成本； $C_f(t)$为t时刻折算到每千瓦时的燃料耗费成本； $C_{ma}(t)$为t时刻各个电能单元折算到每千瓦时的维护管理成本；$C_{dep}(t)$为t时刻各个电力单元的折旧损耗；$kC_{grid}(t)$指t时刻与大电网的交互成本，当从大电网购电时，$k > 0$；当向大电网售电时，$k < 0$。

（三） 以环保性运行为目标的数学模型

参照微电网系统可能产生的污染以及各出力单元可能产生的污染，本文重点考虑排放到空气中的污染气体（CO、 $CO2$、 $SO2$ 以及NOx衍生物）的处理费用[32]，建立的环保性运行成本最低的目标函数公式如下:

$$minC_2 = \sum_{t=1}^{T}\sum_{j=1}^{J}\alpha_j\left(\sum_{i=1}^{N}\beta_{ij}P_{it} + \beta_{mj}P_{mt}\right) \tag{2}$$

式中： C_2 是只考虑微电网环保性运行时的运行成本（元），主要考虑污染气体的治理费用；j表示第j类污染气体（包括微电网和大电网中产生的CO、 $CO2$、 $SO2$ 以及NOx衍生物）； α_j指第j类污染气体的处理费用（元/kw）； β_{ij}和β_{mj}指第j类污染气体的排放量分别在微电网发电单元和大电网中折合到单位功率的排放系数； P_{it}和P_{mt}指t时刻微电网发电单元和大电网当前的实际工作功率(kw).

（四） 以综合效益运行最优为目标的数学模型

由于微电网中像燃料电池发电装置、光伏发电机、风力发电机这样的以清洁可再生能源为能量的微电源，几乎不产生污染气体，符合我们绿色可发展的追求，但要建设这一整套可再生能源发电装置的成本是比较高的，因此在追求环境效益好的同时，控制成本最低是不现实的，本文研究的微电网优化运行的关键点就在于寻找经济性运行和环保性运行间的平衡点，也是将多目标问题转化为单目标的问题，建立的同时考虑的经济性、环保性的以综合效益为目标函数的公式为：

$$\begin{cases} min\ C_3 = [C_1, C_2] \\ C_3 = \lambda_1 C_1 + \lambda_2 C_2 \\ \lambda_1 + \lambda_2 = 1 \end{cases} \tag{3}$$

式中：C1 是考虑微电网经济性运行时的运行成本，C2 是考虑微电网环保性运行时的运行成本，C3 是综合考虑微电网经济性和环保性运行时的运行成本（元），经济性和环保性保持一定的平衡是本文追求的最好的优化结果，本文用线性权重赋值法将经济性和环保性的多目标问题转化为单目标问题。权重系数$\lambda 1$、 $\lambda 2$ 不是由主观猜想得到，也不是计算机随机产生，本文是根据目标函数的变化规律，选取适应函数的值。经试验选取$\lambda 1$=0.43。

三 模拟退火粒子群算法介绍

（一）模拟退火算法

模拟退火算法(Simulated Annealing，SA)思想灵感来源于模拟燃烧固体退火的原理，本质上也是一种基于概率的随机算法。 固体燃烧时内部的颗粒随温度的升高无序运动变多，徐徐加温时，固体内能增大，可达到一个极值，但是当温度徐徐降低时，内部颗粒的无序运动开始减少，渐渐有序状态， 固体内能降为最小，此时就达到了一种相对稳定的状态，也成为平衡态。模仿这种物理现象设计的智能算法被称为“模拟退火算法”。

（二）粒子群算法

粒子群优化算法（Particle Swarm Optimization，PSO）由科学家 Kennedy 和 Eberhart 通过人工生命研究结果启发而提出，核心思想来源于鸟群/鱼群的捕食行为，属于群智能优化算法， 群智能算法一个最大特点就是可以同时搜索待优化目标函数的解空间尽可能多的区域。类似的群智能优化算法还有模拟昆虫、鱼群和兽群等群集行为的蚁群算法(Ant Colony Optimization，ACO)、萤火虫算法(Firefly Algorithm，FA)等，这些算法都具有群体合作觅食这一共性，且群体中每个成员都会学习自身的经验以及群体的经验来不断探寻事物的位置，帮助种群找到食物。

粒子群优化算法是通过模拟鸟群在空中飞行的觅食行为，将鸟与鸟群之间彼此协作的方式达到最优目的即找到食物。每个“粒子”被想象成一只鸟，是每个寻优问题的解，在 D 维空间进行搜索。由被命名为 fitness function 这样的函数确定适应环境的适应度值来判断当前位置所有粒子的好坏。每一个粒子都具有学习的能力，可以记住之前搜寻过程中碰到的最佳位置。每个粒子都有各自的速度矢量来决定飞行的距离和方向，这个速度矢量会根据本身的觅食经验以及从群体获得的信息来进行实时的调整，这也是粒子群算法中“进化”的含义。也就是说，每个粒子（每只鸟） 都具有四个属性，即位置、经验、速度、适应度，为了更好的理解，可以这样认为：位置代表函数的自变量值；经验代表自身经过的距离食物最近的位置；速度代表自变量的变化值；适应度代表距离食物的位置，也就是函数值。这样就可以将实际问题抽象转化成数学函数问题，而本质上就是利用粒子群算法求解函数的最值问题。

（三）改进的模拟退火粒子群算法

根据上述两节的说明，为了改进容易陷入局部解的粒子群算法，使之更适用于研究微电网优化运行中，本文提出了融合粒子群算法和模拟退火算法的模拟退火粒子群算法(Simulated Annealing Particle Swarm Optimization，SAPSO)，融合的方法主要是引入了模拟退火算法中的概率突跳的思想， 也就是使 SAPSO 算法不仅仅大概率能接受更好的解，也有一定概率尝试接受一些差一些的解从而跳出局部最优，考虑更多种解的可能性，使粒子种群的运动行为更加合理，从而得到更好的微电网运行方式。

（1） 改进的模拟退火粒子群算法实现步骤

（a） 初始化参数：包括惯性权重ω、学习因子c_1、 c_2和退火速度δ。

（b） 随机生成解域中一个种群，包含 m 个粒子，随机初始化粒子各个的速度和位置。

（c） 计算每个粒子i的适应度$fitness(x(i))$， $fitness(x(i))$为适应度函数也就是目标函数，简称$f(x)$，记录每个（第i个）粒子的位置P_{id}、全局最优位置P_{pd}、适应度$f(P_{id})$和全局最优适应度$f(P_{pd})$。

（d） 根据全局最优适应度$f(P_{pd})$计算得到退火算法初始温度T。

（e） 记f_{SA}(x)函数为计算退火算法适应度的函数，在当前温度T下各P_{id}的退火算法适应度

$$f_{SA}(P_{id}) = \frac{e^{-\frac{f(P_{id})-f(P_{pd})}{T}}}{\sum_{i=1}^{n} e^{-\frac{f(P_{id})-f(P_{pd})}{T}}} \tag{4}$$

（f） 应用轮盘赌选择算法的思想，从个体最优位置Pid中选取一个代替全局最优位置P_{pd}，记为P_{rd}。其中：轮盘赌选择方法是选择算子的一种，又称比例选择方法，其基本思想是：用概率的方法将种群中适应度表现良好的个体选出，各个个体适应度越高被选择的概率越高。本文中，该算法思想如下：

1) $pBet = rand()$ ($pBet$为一个 0 到 1 的随机数)

2) 根据退火算法适应度$fSA(Pid)$计算累积概率comfit(j) $= \sum_{i=1}^{j} f_{SA}(P_{id})$

3) 根据累积概率，选用满足条件的第 r 个粒子的个体最优位置P_{rd}代替全局最优位置P_{pd}。条件为： $comfit(r-1) < pBet < comfit(r)$

（g） 将P_{pd}替换成P_{rd}，带入粒子群公式，更新各粒子的速度如下：

$$v_{id}^{k+1} = \backslash omega v_{id}^{k} + c_1 r_1 (p_{id}^{k} - x_{id}^{k}) + c_2 r_2 (p_{rd}^{k} - x_{id}^{k}) \tag{5}$$

（h） 再计算各个粒子的适应度，更新各粒子的最优位置P_{id}和种群最优位置P_{pd}。

（i） 进行退火操作：T = δt

（j） 判断是否满足提前设置好的终止条件，若满足则停止搜索，输出返回值，若不满足则转至步骤(e)。

（2） 改进的模拟退火粒子群算法实现如图 2

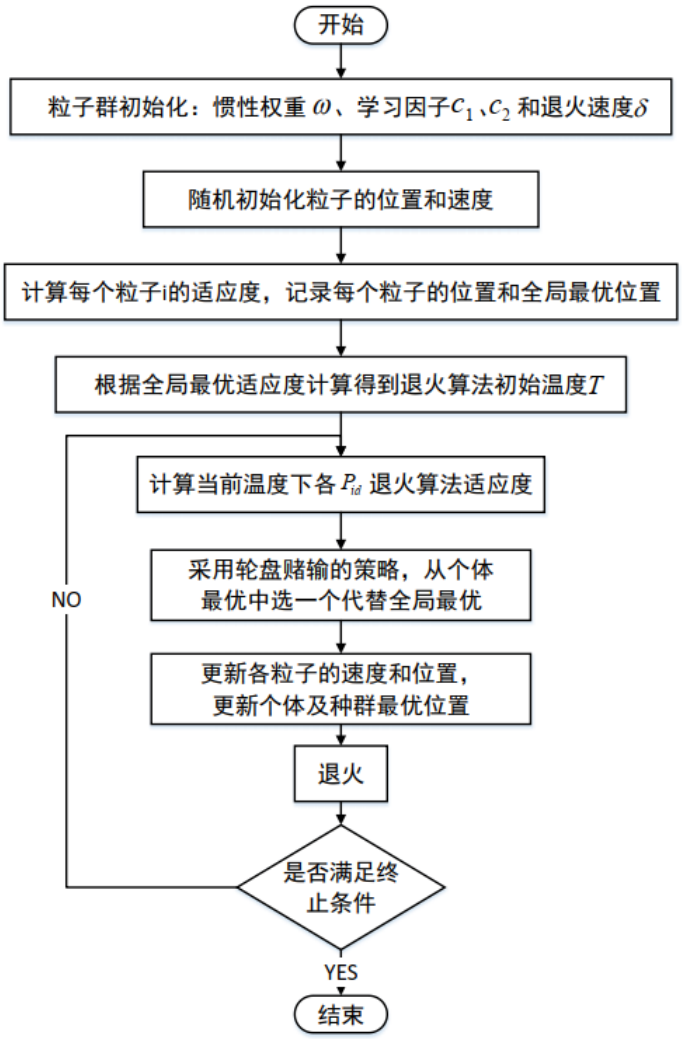


图 2 改进的模拟退火粒子群算法实现流程图

四 问题的设计及结果分析

（一）模拟退火粒子群算法求解的思路和流程

微电网优化调度中的具体步骤如下：

（1） 初始化微电网参数 WIP，PV，WT，MT，FC，BAT。

（2） 初始化模拟退火粒子群的位置和速度。

（3） 计算粒子的适应度并按 T0=-│ΔCT│/log(0.2)初始化退火温度，其中ΔCT=ΔCO+ΔCE−ΔCN。

（4） 将每个粒子的适应度当做粒子个体最优极值 pbest，在个体极值中选择最优值当做群体极值 gbest。

（5） 确定算法是否达到最大迭代次数。若是，则输出微电网优化调度结果，否则执行 R 从 1 到 M 循环部分，即 R→R+1，其中 M 为最大迭代次数。

（6） 计算每个粒子适应度 CT(R)和平均适应度 CTa(R)。

（7） 若粒子的适应度优于原有个体极值 pbest，则当前适应度设置为 pbest，选择最优个体极值作为群体极值 gbest。

（8） 对每次迭代过程中小于平均适应度的粒子位置按式 3.25 进行混沌扰动，按式 3.21-3.22 更新粒子的速度和位置。

（9） 计算每个混沌扰动后粒子的适应度 CT(R+1)和平均适应度 CTa(R+1)。

（10） 计算 2 个位置适应度的变化量ΔCT=CT(R+1)-CT(R)，如果ΔCT<0 或按 Metropolis 准则 exp(-ΔCT/T)>rand，其中 rand 为 0~1 之间的随机数，则接受新位置，否则保留旧位置。

（11） 按 T=λT 执行退温操作，其中 λ 为模拟退火过程中的退温系数。改进模拟退火粒子群算法对微电网优化调度模型求解流程如图 3

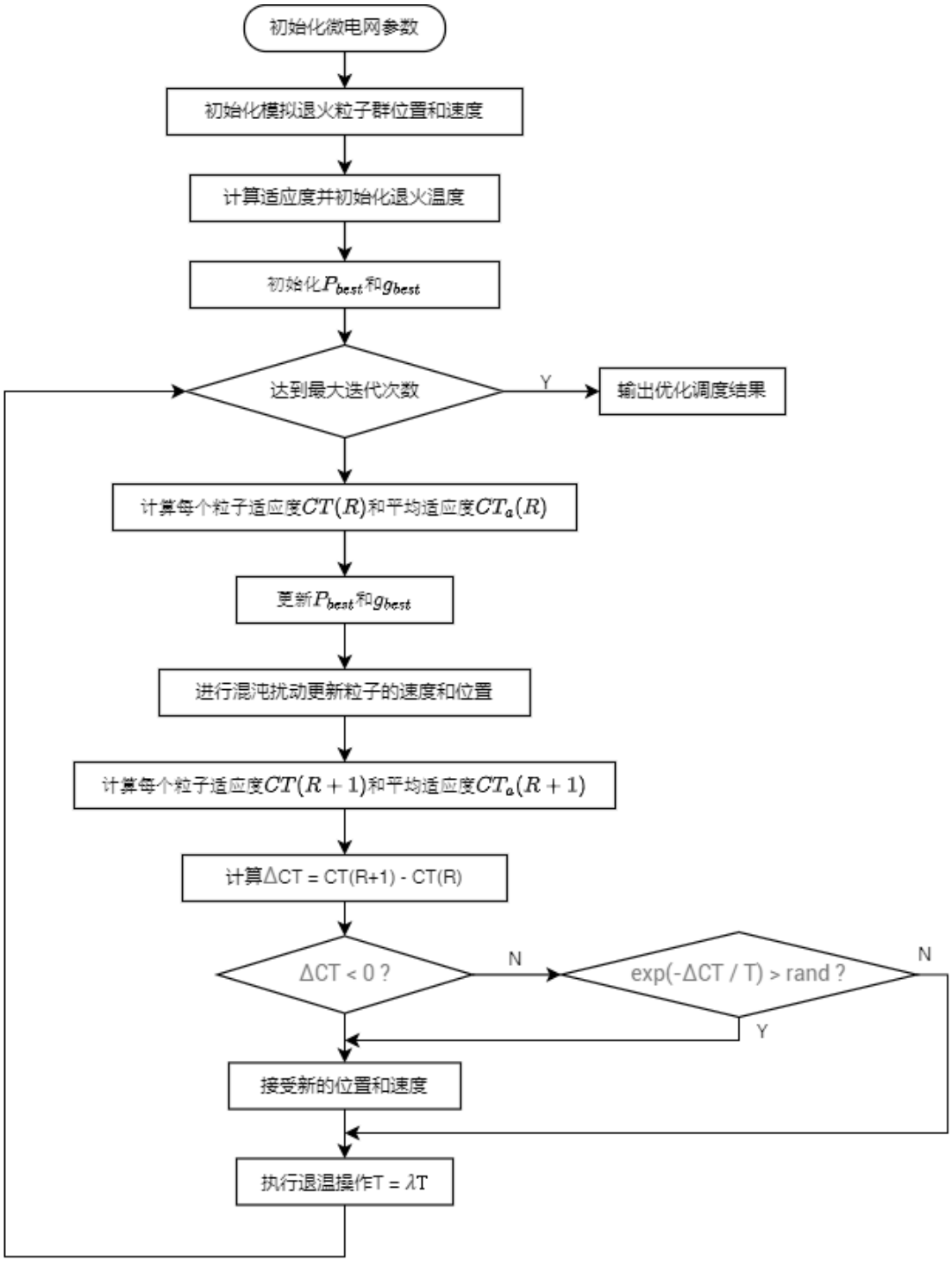


图 3 改进模拟退火粒子群算法对微电网优化调度模型求解流程图

（二） 问题的求解和结果分析

（1） 参数范围及初始化过程

设置初始参数如下：

微型燃气轮机最大功率 MTMaxPower=5;

微型燃气轮机最小功率 MTMinPower=1;

电网输入微网最大功率 GridMaxImportPower=250;

电网输入微网最小功率 GridMinImportPower=10;

储能最大放电功率 StorageMaxDischargingPower=150;

储能最大充电功率 StorageMaxChargingPower=-100;

（2） 模拟退火粒子群迭代取优过程

```
%计算各个粒子适应度
for i=1:N
    p(i)=fitness(x(i,:),s);
    y(i,:)=x(i,:);%每个粒子的个体寻优值
end
Pbest=fitness(x(1,:),s);
pg=x(1,:);%Pg 为全局最优
for i=2:N
    if fitness(x(i,:),s)<fitness(pg,s)
        Pbest=fitness(x(i,:),s);
        pg=x(i,:);%全局最优更新
    end
end
```

（3） 结果处理与花费处理

花费方程 matlab 函数实现：

```
function cost=economic(x)
C_MT=0;C_GRID=0;C_BA=0;
G_price=[0 0 0 0 0 0 0 0 0.0931 0.0931 0.0931 0.0931 0.0931 0.0931 0.0931 0.0931
0.0931 0.0931 0.0931 0.0931 0.0931 0 0 0];
for i=1:72
    if i<25
      C_MT=C_MT+0.04*x(i);
    elseif i>24&&i<49
      C_GRID= C_GRID+G_price(i-24)*x(i);
    else
      C_BA=C_BA+8*abs(x(i));
```

```
    end
end
cost=C_MT+C_GRID+C_BA;
```

问题结果 matlab 函数实现：

```
function result=fitness(x,k)
C_MT=0;C_GRID=0;C_BA=0;P_BA_sum=0;deltp_sum=0;P_BA_sum_delt=0;BAsocMax=200;
P_load=[11.7 12.4 11.7 12.4 11.7 22.4 81.9 122.4 241.3 242.0 241.3 241.3 241.3
240.7 241.3 240.7 241.3 161.0 122.2 121.1 102.9 101.2 23.0 11.1];
P_pv=[0 0 0 0 0 0 0 0.0391 19.5244 40.0204 50.1010 73.3946 74.3511 76.3489 72.8004
53.1185 4.6258 0 0 0 0 0 0 0];
G_price=[0 0 0 0 0 0 0 0 0.0931 0.0931 0.0931 0.0931 0.0931 0.0931 0.0931 0.0931
0.0931 0.0931 0.0931 0.0931 0.0931 0 0 0];
for i=1:72
    if i<25
      C_MT=C_MT+0.04*x(i);
    elseif i>24&&i<49
      C_GRID= C_GRID+G_price(i-24)*x(i);
    else
      C_BA=C_BA+8*abs(x(i));
    end
end
for i=49:72
      P_BA_sum=P_BA_sum+(-x(i));
      P_BA_sum_delt= P_BA_sum_delt+max(0,P_BA_sum-BAsocMax);
end
if(P_BA_sum_delt<=0)
    d=0;
elseif(P_BA_sum_delt>0&&P_BA_sum_delt<=0.001)
   d=10;
elseif(P_BA_sum_delt>0.001&&P_BA_sum_delt<=0.1)
    d=20;
elseif(P_BA_sum_delt>0.1&&P_BA_sum_delt<=1.0)
    d=100;
else
    d=1000;
end
for i=1:24
   deltp(i)=abs(P_pv(i)+x(i)+x(i+24)+x(i+48)-P_load(i));
   deltp_sum=deltp_sum+deltp(i);
end
if(deltp_sum==0)
   r=0.0;
elseif(deltp_sum>0&&deltp_sum<=0.001)
    r=10;
```

```
elseif(deltp_sum>0.001&&deltp_sum<=0.1)
   r=20;
elseif(deltp_sum>0.1&&deltp_sum<=1.0)
   r=100;
else
   r=2000;
end
 result=C_MT+C_GRID+C_BA+d*P_BA_sum_delt+r*deltp_sum;
```

（4） 分析结果与结论

最后得到的优化极值为：

Result = 5.751070872694864e+03

Resulta =9.747599669821131e+04

s =300%在温度最高时获取最由极值

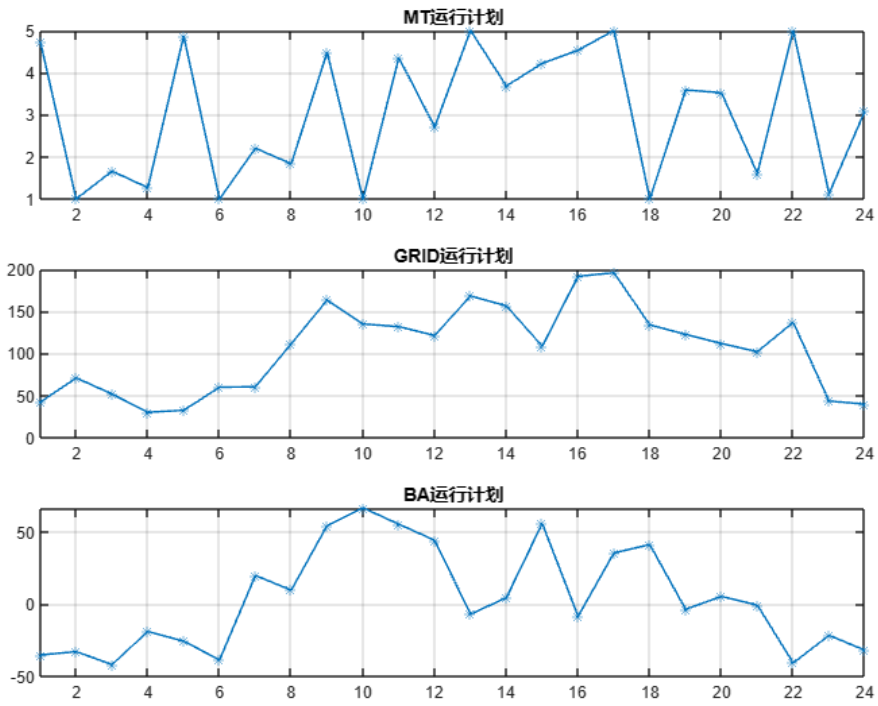


图 4 运行计划与时间关系图

五 实验总结

本文研究如何在保持微电网系统安全稳定运行以及满足日常负荷的条件下，优化可控型微电源的出力、储能装置的充放电以及与公共电网的交互功率，达到微电网经济效益和环保效益同时最优的双目标问题。结合现代智能算法，即粒子群算法、模拟退火算法，将其融合为一种基于模拟退火粒子群的融合算法，通过围绕局部最优解，全局最优解等算法关键元素，得到一种具有这两种算法

优点的算法实现方式。在本次实验中我们了解到了粒子群算法、模拟退火算法的真正的含义，从而对这两种算法有了更好的认识。通过输出结果，我们能对问题更好一步掌握，从而达到更好的效果。对于之后的学习，我们也要继续保持一颗探寻知识，搞懂知识的心，才能在算法中不断学习，不断进步。

参考文献

[1]占缘.基于改进模拟退火粒子群算法的微电网优化调度.2021.南昌大学,MA thesis.

[2]李书益.基于遗传模拟退火粒子群算法的微电网优化运行研究.2020.南京邮电大学,MA thesis.

[3]葛显龙, 王伟鑫,and 李顺勇.智能算法及应用.西南交通大学出版社.

[4]鲍永胜, 孙洁,and 宋鸿雁."混合粒子群优化算法的新能源电网规划研究." 能源与环保 44.11(2022):147-153. doi:10.19389/j.cnki.1003-0506.2022.11.025.

附录

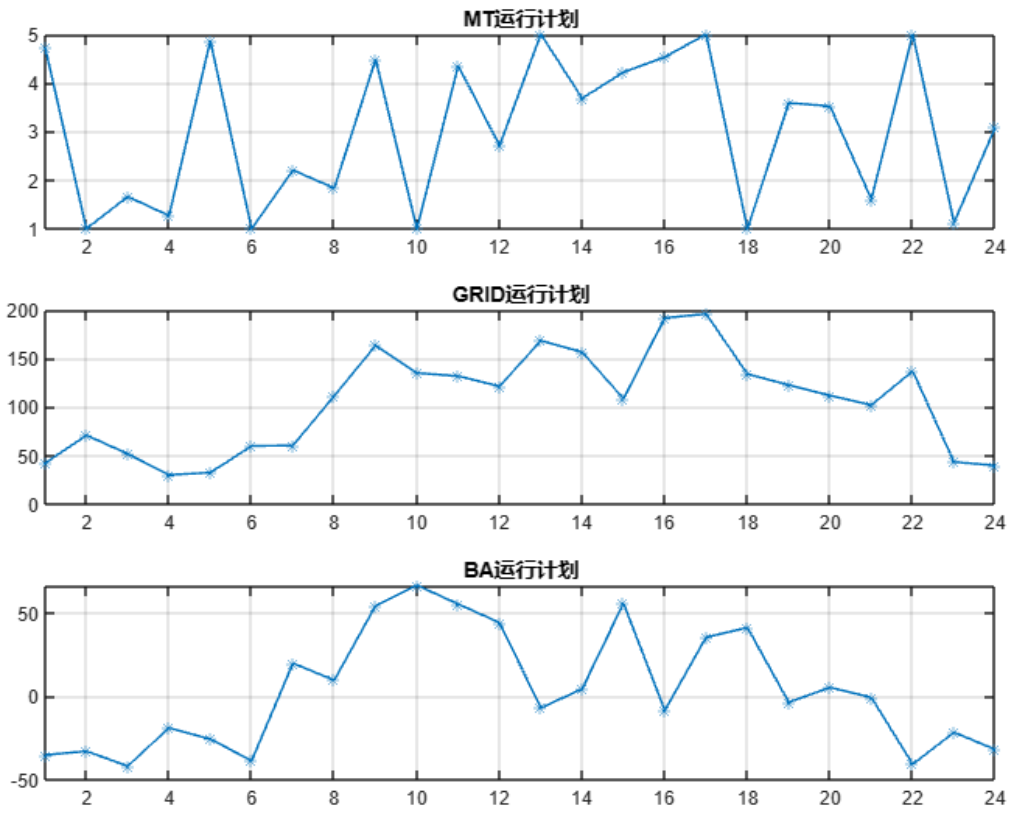
```
命令行窗口
*****************************
函数的全局最优位置为:

Solution =

   1.0e+02 *

   0.047358743678479
   0.010000126787232
   0.016606321684647
   0.012693127800473
   0.048664980275662
   0.010000694271496
   0.022047141873036
   0.018343135325399
   0.044811176328292
   0.010000156173475
   0.043532440494206
   0.027086621449666
   0.049999579019306
   0.036811993861594
   0.042190085039501
   0.045285877039111
   0.049998224788206
   0.010000691902876
   0.035876445663378
   0.035203828762094
   0.016047382506804
   0.049999697027235
   0.011273518518458
   0.030871857043279
   0.421084799829765
   0.709826309448851
   0.519005501555009
   0.302419626161130
   0.325231245473080
   0.599489528055527
   0.604634344602892
```

```
命令行窗口
   1.212829687109876
   1.681414032695340
   1.564409655712548
   1.085761127802674
   1.914925479159798
   1.956608625087045
   1.338689869675137
   1.225981681249394
   1.117708664405007
   1.020583448738047
   1.368104840953833
   0.434941238462562
   0.399156411873756
  -0.351546851289619
  -0.328663173497811
  -0.417626797942527
  -0.189682523021674
  -0.256896749118404
  -0.385404115552917
   0.196526820797324
   0.095866983541663
   0.541060331713969
   0.663243978987611
   0.550501275458169
   0.439584976370994
  -0.069860178685227
   0.043187632075664
   0.555760406623855
  -0.084967150645869
   0.352437477881127
   0.409752600880916
  -0.036413337621727
   0.052255690406847
  -0.007631928740103
  -0.406741564692197
  -0.216211809246059
  -0.319052862596986
```

迭代数据

```
  -0.216211809246059
  -0.319052862596986

最后得到的优化极值为:

Result =

     5.751070872694864e+03


Resulta =

     9.747599669821131e+04


s =

   300

*****************************************************************
>>
```

运行最优解



迭代曲线

```
function cost=economic(x)
C_MT=0;
C_GRID=0;
C_BA=0;
G_price=[0 0 0 0 0 0 0 0 0.0931 0.0931 0.0931 0.0931 0.0931 0.0931 0.0931 0.0931 0.0931 0.0931 0.0931 0.0931 0.0931 0 0 0];
for i=1:72
    if i<25
      C_MT=C_MT+0.04*x(i);
    elseif i>24&&i<49
      C_GRID= C_GRID+G_price(i-24)*x(i);
    else
      C_BA=C_BA+8*abs(x(i));
    end
end
cost=C_MT+C_GRID+C_BA;

function result=fitness(x,k)
C_MT=0;
C_GRID=0;
C_BA=0;
P_BA_sum=0;
deltp_sum=0;
P_BA_sum_delt=0;
BAsocMax=200;
P_load=[11.7 12.4 11.7 12.4 11.7 22.4 81.9 122.4 241.3 242.0 241.3 241.3 241.3 240.7 241.3 240.7 241.3 161.0 122.2 121.1 102.9 101.2 23.0 11.1];
P_pv=[0 0 0 0 0 0 0 0.0391 19.5244 40.0204 50.1010 73.3946 74.3511 76.3489 72.8004 53.1185 4.6258 0 0 0 0 0 0 0];
G_price=[0 0 0 0 0 0 0 0 0.0931 0.0931 0.0931 0.0931 0.0931 0.0931 0.0931 0.0931 0.0931 0.0931 0.0931 0.0931 0.0931 0 0 0];

for i=1:72
    if i<25
      C_MT=C_MT+0.04*x(i);
    elseif i>24&&i<49
      C_GRID= C_GRID+G_price(i-24)*x(i);
    else
      C_BA=C_BA+8*abs(x(i));
    end
end

for i=49:72
      P_BA_sum=P_BA_sum+(-x(i));
```

```
        P_BA_sum_delt= P_BA_sum_delt+max(0,P_BA_sum-BAsocMax);
end


if(P_BA_sum_delt<=0)
    d=0;
elseif(P_BA_sum_delt>0&&P_BA_sum_delt<=0.001)
   d=10;
elseif(P_BA_sum_delt>0.001&&P_BA_sum_delt<=0.1)
    d=20;
elseif(P_BA_sum_delt>0.1&&P_BA_sum_delt<=1.0)
    d=100;
else
    d=1000;
end

for i=1:24
   deltp(i)=abs(P_pv(i)+x(i)+x(i+24)+x(i+48)-P_load(i));
   deltp_sum=deltp_sum+deltp(i);
end

if(deltp_sum==0)
   r=0.0;
elseif(deltp_sum>0&&deltp_sum<=0.001)
%    r=10*sqrt(k);
   r=10;
elseif(deltp_sum>0.001&&deltp_sum<=0.1)
   r=20;
elseif(deltp_sum>0.1&&deltp_sum<=1.0)
   r=100;
else
   r=2000;
end
result=C_MT+C_GRID+C_BA+d*P_BA_sum_delt+r*deltp_sum;


function result=fitness(x,k)
C_MT=0;
C_GRID=0;
C_BA=0;
P_BA_sum=0;
deltp_sum=0;
P_BA_sum_delt=0;
```

```
BAsocMax=200;
P_load=[11.7 12.4 11.7 12.4 11.7 22.4 81.9 122.4 241.3 242.0 241.3 241.3 241.3 240.7 241.3 240.7 241.3 161.0 122.2 121.1 102.9 101.2 23.0 11.1];
P_pv=[0 0 0 0 0 0 0 0.0391 19.5244 40.0204 50.1010 73.3946 74.3511 76.3489 72.8004 53.1185 4.6258 0 0 0 0 0 0 0];
G_price=[0 0 0 0 0 0 0 0 0.0931 0.0931 0.0931 0.0931 0.0931 0.0931 0.0931 0.0931 0.0931 0.0931 0.0931 0.0931 0 0 0];

for i=1:72
    if i<25
      C_MT=C_MT+0.04*x(i);
    elseif i>24&&i<49
      C_GRID= C_GRID+G_price(i-24)*x(i);
    else
      C_BA=C_BA+8*abs(x(i));
    end
end

for i=49:72
      P_BA_sum=P_BA_sum+(-x(i));
      P_BA_sum_delt= P_BA_sum_delt+max(0,P_BA_sum-BAsocMax);
end


if(P_BA_sum_delt<=0)
    d=0;
elseif(P_BA_sum_delt>0&&P_BA_sum_delt<=0.001)
   d=10;
elseif(P_BA_sum_delt>0.001&&P_BA_sum_delt<=0.1)
    d=20;
elseif(P_BA_sum_delt>0.1&&P_BA_sum_delt<=1.0)
    d=100;
else
    d=1000;
end

for i=1:24
   deltp(i)=abs(P_pv(i)+x(i)+x(i+24)+x(i+48)-P_load(i));
   deltp_sum=deltp_sum+deltp(i);
end

if(deltp_sum==0)
   r=0.0;
elseif(deltp_sum>0&&deltp_sum<=0.001)
```

```
%    r=10*sqrt(k);%%%%µü´ú´ÎÊý
   r=10;
elseif(deltp_sum>0.001&&deltp_sum<=0.1)
   r=20;
elseif(deltp_sum>0.1&&deltp_sum<=1.0)
   r=100;
else
   r=2000;
end
result=C_MT+C_GRID+C_BA+d*P_BA_sum_delt+r*deltp_sum;


clear
clc
format long;

%初始化条件
%微型燃气轮机最大功率
MTMaxPower=5;
%微型燃气轮机最小功率
MTMinPower=1;
%电网输入微网最大功率
GridMaxImportPower=250;
%电网输入微网最小功率
GridMinImportPower=10;
%储能最大放电功率
StorageMaxDischargingPower=150;
%储能最大充电功率
StorageMaxChargingPower=-100;

Max_Dt=300;%最大迭代次数 300
D=72;%搜索空间维数（未知数个数）
N=600;%粒子个数 600
w_max=0.9;
w_min=0.4;
v_max=2;
s=1;

%初始化种群个体（位置和速度）
for i=1:N
    for j=1:72
%         v(i,j)=randn;
        v(i,j)=0.0;
        if j<25
```

```
            x(i,j)=MTMinPower+rand()*(MTMaxPower-MTMinPower);
        elseif j>24&&j<49
            x(i,j)=GridMinImportPower+rand()*(GridMaxImportPower-GridMinImportPower);
        elseif j>48&&j<73
            x(i,j)=StorageMaxChargingPower+rand()*(StorageMaxDischargingPower-StorageMaxChargingPower);
        end
    end
end

%计算各个粒子的适应度，并初始化 Pi 和 Pg
for i=1:N
    p(i)=fitness(x(i,:),s);
    y(i,:)=x(i,:);%每个粒子的个体寻优值
end
Pbest=fitness(x(1,:),s);
pg=x(1,:);%Pg 为全局最优
for i=2:N
    if fitness(x(i,:),s)<fitness(pg,s)
       Pbest=fitness(x(i,:),s);
       pg=x(i,:);%全局最优更新
    end
end

%进入主循环
for t=1:Max_Dt
    for i=1:N
        w=w_max-(w_max-w_min)*t/Max_Dt;%惯性权重更新
        c1=(0.5-2.5)*t/Max_Dt+2.5; %认知
        c2=(2.5-0.5)*t/Max_Dt+0.5; %社会认识
%         w=0.7;
%         c1=2.05; %认知
%         c2=2.05; %社会认识
        v(i,:)=w*v(i,:)+c1*rand()*(y(i,:)-x(i,:))+c2*rand()*(pg-x(i,:));
        for m=1:72
            if(v(i,m)>v_max)
                v(i,m)=v_max;
            elseif(v(i,m)<-v_max)
                v(i,m)=-v_max;
            end
        end

        x(i,:)=x(i,:)+v(i,:);
```

```
        %对粒子边界处理
        for n=1:72
            if n<25
                    if x(i,n)<MTMinPower
                          x(i,n)=MTMinPower;
                          v(i,n)=-v(i,n);
                    elseif x(i,n)>MTMaxPower
                          x(i,n)=MTMaxPower;
                          v(i,n)=-v(i,n);
                    else
                          delt(i,n)=0;
                    end
            elseif n>24&&n<49
                     if x(i,n)<GridMinImportPower
                          x(i,n)=GridMinImportPower;
                          v(i,n)=-v(i,n);
                    elseif  x(i,n)>GridMaxImportPower
                          x(i,n)=GridMaxImportPower;
                          v(i,n)=-v(i,n);
                     else
                          delt(i,n)=0;
                     end
            else
                     if x(i,n)<StorageMaxChargingPower
                           x(i,n)=StorageMaxChargingPower;
                           v(i,n)=-v(i,n);
                    elseif x(i,n)>StorageMaxDischargingPower
                         x(i,n)=StorageMaxDischargingPower;
                         v(i,n)=-v(i,n);
                     else
                         delt(i,n)=0;
                     end
            end
        end
        %对粒子进行评价，寻找最优值
        if fitness(x(i,:),t)<p(i)
            p(i)=fitness(x(i,:),t);
            y(i,:)=x(i,:);
        end
        if p(i)<Pbest
            Pbest=p(i);
            pg=y(i,:);
            s=t;
        end
```

```
    end
    %Pbest(t)=fitness(pg,s);
end

disp('函数的全局最优位置为：')

Solution=pg'
for m=1:24
    pg1(m)=pg(m);
end
for m=25:48
    pg2(m-24)=pg(m);
end
for m=49:72
    pg3(m-48)=pg(m);
end
figure
subplot(311)
plot( pg1,'-*')
xlim([1 24])
grid
title('MT 运行计划')
subplot(312)
plot( pg2,'-*')
xlim([1 24])
grid
title('GRID 运行计划')
subplot(313)
plot( pg3,'-*')
xlim([1 24])
grid
title('BA 运行计划')
disp('最后得到的优化极值为：')

Result=economic(pg)
Resulta=fitness(pg,s)
s
```